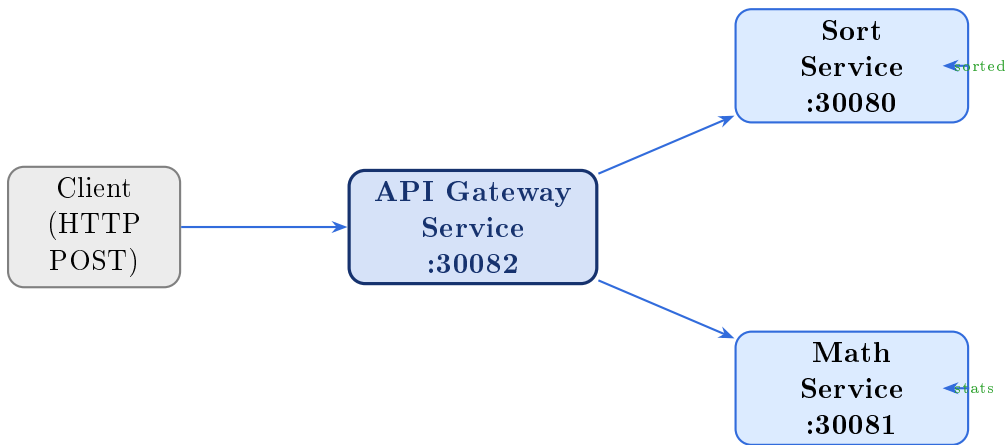

Microservices Orchestration with Kubernetes

Full Technical Documentation



Project: Microservices Orchestration with Kubernetes
Group: 04
Institution: ENSA Tétouan
Contact: G04@ensa-tetouan.ma
DockerHub: marouaneloua
Version: 1.0
Date: May 2026

Contents

1	Project Overview	2
1.1	Introduction	2
1.2	Goals and Objectives	2
1.3	Technology Stack	3
2	System Architecture	4
2.1	High-Level Architecture	4
2.2	Kubernetes Object Model	5
2.3	Service Discovery	5
3	Microservices — Source Code	6
3.1	Sort Service	6
3.1.1	Responsibility	6
3.1.2	Source Code	6
3.1.3	Dockerfile	7
3.2	Math Service	8
3.2.1	Responsibility	8
3.2.2	Computed Statistics	8
3.2.3	Source Code	8
3.2.4	Dockerfile	9
3.3	API Gateway Service	10
3.3.1	Responsibility	10
3.3.2	Source Code	11
3.3.3	Dockerfile	12
4	Kubernetes Configuration Files	14
4.1	Sort Service — Deployment & Service (<code>deployment.yml</code> and <code>service.yml</code>)	14
4.1.1	Deployment	14
4.1.2	Service	15
4.1.3	Resource Limits Explained	15
4.2	Math Service — Deployment & Service (<code>math-deployment.yml</code>)	15
4.3	Gateway Service — Deployment & Service (<code>gateway-deployment.yml</code>)	16
5	Docker Images & DockerHub	18
5.1	Published Images	18
5.2	Building Images Locally (Minikube)	18
5.3	Building and Pushing to DockerHub	18
6	Deployment Guide	20
6.1	Prerequisites	20
6.2	Step-by-Step Deployment	20

6.2.1	Step 1 — Start Minikube	20
6.2.2	Step 2 — Option A: Build Images in Minikube (local)	20
6.2.3	Step 2 — Option B: Pull Images from DockerHub	20
6.2.4	Step 3 — Apply All Kubernetes Manifests	21
6.2.5	Step 4 — Verify Deployment	21
6.2.6	Step 5 — Get the Gateway URL	21
6.2.7	Step 6 — Test the API	21
6.3	Tearing Down	22
7	Kubernetes Features in Use	23
7.1	High Availability via Replicas	23
7.2	Rolling Updates (Zero Downtime)	23
7.3	Resource Quotas	23
7.4	Internal DNS-Based Service Discovery	23
7.5	NodePort Services	24
8	Security Considerations	25
8.1	Non-Root Container Execution	25
8.2	Minimal Base Image	25
8.3	No External Dependencies	25
8.4	Internal-Only Back-End Services	25
9	Project File Structure	26
10	Troubleshooting	27
10.1	Common Issues	27
10.1.1	Pods in ImagePullBackOff	27
10.1.2	Gateway Returns 502	27
10.1.3	Minikube Tunnel Not Accessible	27
10.2	Useful Diagnostic Commands	27
11	Summary	29

Chapter 1

Project Overview

1.1 Introduction

This project, developed by **Group 04** at ENSA Tétouan, demonstrates the design, containerisation, and orchestration of a **microservices-based application** using **Kubernetes**. The system accepts a list of numbers through a single HTTP entry-point and returns both a sorted version of that list and a set of statistical descriptors.

The architecture follows the *API Gateway* pattern: a single public-facing service dispatches requests to two independent back-end microservices, each responsible for one well-defined computation. All components are containerised with Docker and managed by Kubernetes, enabling high availability, declarative configuration, and rolling updates.

1.2 Goals and Objectives

1. Apply the microservices architectural pattern to a concrete problem.
2. Containerise each service with an optimised, security-hardened Docker image.
3. Deploy and orchestrate the services on a Kubernetes cluster (Minikube).
4. Demonstrate Kubernetes features: Deployments, Services, rolling updates, resource limits, and internal DNS-based service discovery.
5. Publish images to DockerHub for easy distribution.

1.3 Technology Stack

Technology	Role
Python 3.11	Runtime for all three microservices
Docker	Containerisation of each service
Kubernetes (Minikube)	Local cluster orchestration
kubectl	Cluster management CLI
DockerHub	Public image registry (<code>marouaneloua</code>)
<code>http.server</code>	Lightweight built-in HTTP server (no external deps)

Table 1.1: Technology stack

Chapter 2

System Architecture

2.1 High-Level Architecture

The system is composed of **three microservices**, all running as Kubernetes Deployments inside the same cluster namespace:

API Gateway Service

Public entry-point exposed via NodePort 30082. Receives client requests, fans them out to the two back-end services, and aggregates the results into a single JSON response.

Sort Service

Internal-only service on ClusterIP port 80 (container port 8080). Sorts the input list using Python's built-in `sorted()` function.

Math Service

Internal-only service on ClusterIP port 81 (container port 8081). Computes descriptive statistics: count, min, max, mean, and median.

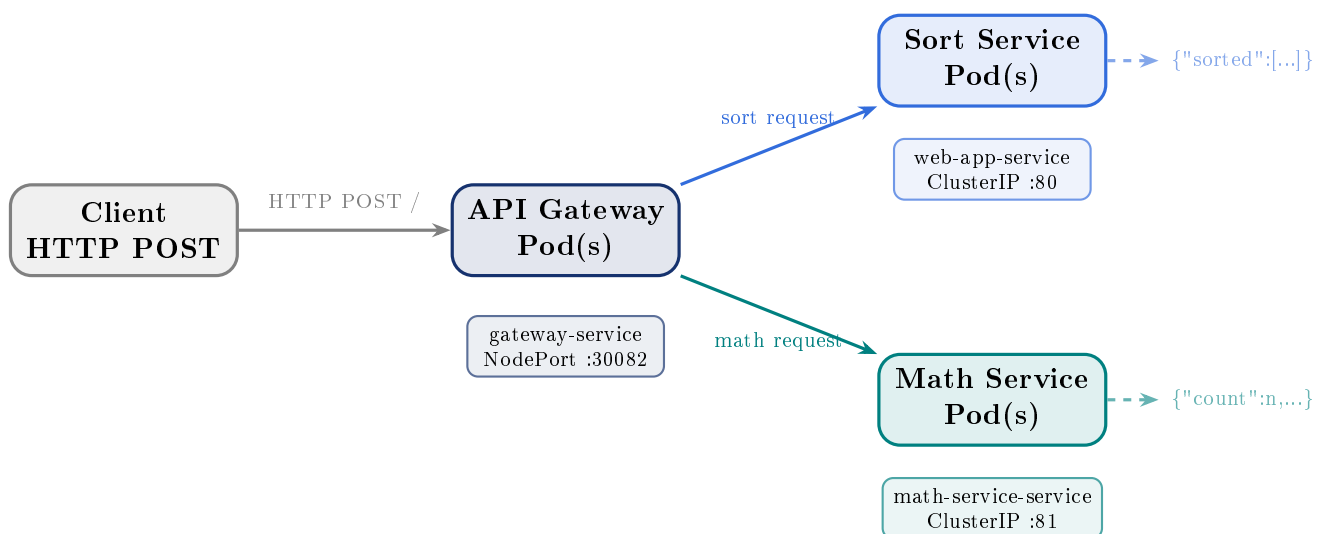


Figure 2.1: Request flow through the Kubernetes cluster

2.2 Kubernetes Object Model

Service	Deployment	Replicas	Service Type	Ports (node:svc:pod)
Sort Service	web-app-deployment	2	NodePort	30080 : 80 : 8080
Math Service	math-service-deployment	2	NodePort	30081 : 81 : 8081
Gateway Service	gateway-deployment	2	NodePort	30082 : 82 : 8000

Table 2.1: Kubernetes objects summary

2.3 Service Discovery

Kubernetes provides automatic DNS resolution inside the cluster. The Gateway service communicates with back-end services using their Kubernetes DNS names, avoiding hardcoded IP addresses:

DNS Name	Resolves to
web-app-service:80	Sort Service ClusterIP
math-service-service:81	Math Service ClusterIP

Chapter 3

Microservices — Source Code

3.1 Sort Service

3.1.1 Responsibility

The Sort Service receives a JSON array of numbers and returns the array sorted in ascending order using Python's native `sorted()` function (Timsort — $O(n \log n)$ worst case).

API Contract

Endpoint: POST / **Port:** 8080 (container) / 80 (cluster) / 30080 (node)

Request body:

```
{ "data": [5, 3, 8, 1, 2] }
```

Response body (HTTP 200):

```
{ "sorted": [1, 2, 3, 5, 8] }
```

3.1.2 Source Code

```
1 #!/usr/bin/env python3
2 """Microservice Python - Data Sorting via HTTP."""
3 from http.server import HTTPServer, BaseHTTPRequestHandler
4 import json
5
6 class SortHandler(BaseHTTPRequestHandler):
7     def do_POST(self):
8         length = int(self.headers.get('Content-Length', 0))
9         body = json.loads(self.rfile.read(length))
10        data = body.get('data', [])
11        result = sorted(data) # Python native sorting algorithm
12
13        self.send_response(200)
```



```
14         self.send_header('Content-Type', 'application/json')
15         self.end_headers()
16         self.wfile.write(json.dumps({"sorted":
17                                     result}).encode())
18
19     def log_message(self, format, *args):
20         pass # Silence HTTP logs
21
22 if __name__ == '__main__':
23     server = HTTPServer(('0.0.0.0', 8080), SortHandler)
24     print('Sort Service started on port 8080')
25     server.serve_forever()
```

Listing 3.1: sort_service.py

3.1.3 Dockerfile

```
1 # Step 1: Base Image
2 FROM python:3.11-slim
3
4 # Step 2: Metadata
5 LABEL maintainer='G04@ensa-tetouan.ma'
6 LABEL version='1.0'
7 LABEL description='Microservice de tri Python'
8
9 # Step 3: Working Directory
10 WORKDIR /app
11
12 # Step 4: Dependencies
13 COPY requirements.txt .
14 RUN pip install --no-cache-dir -r requirements.txt && pip cache
15     purge
16
17 # Step 5: Copy Source Code
18 COPY sort_service.py .
19
20 # Step 6: Expose Port
21 EXPOSE 8080
22
23 # Step 7: Non-root User for Security
24 RUN useradd -m appuser && chown -R appuser /app
25 USER appuser
26
27 # Step 8: Start Command
28 CMD ["python", "sort_service.py"]
```

Listing 3.2: Sort Service Dockerfile

Security note: The image runs as a non-root user `appuser`, following Docker security best practices. The `python:3.11-slim` base image minimises the attack

```
surface.
```

3.2 Math Service

3.2.1 Responsibility

The Math Service computes five descriptive statistics from a list of numbers: count, minimum, maximum, mean (moyenne), and median (médiane).

API Contract

Endpoint: POST / **Port:** 8081 (container) / 81 (cluster) / 30081 (node)

Request body:

```
{ "data": [5, 3, 8, 1, 2] }
```

Response body (HTTP 200):

```
{
  "count": 5,
  "min": 1,
  "max": 8,
  "moyenne": 3.8,
  "mediane": 3.0
}
```

Error response (HTTP 400):

```
{ "error": "La liste de donnees est vide" }
```

3.2.2 Computed Statistics

Field	Formula	Description
count	n	Number of elements
min	$\min(x_i)$	Smallest element
max	$\max(x_i)$	Largest element
moyenne	$\bar{x} = \frac{1}{n} \sum x_i$	Arithmetic mean
mediane	Middle value of sorted list	Median (avg of middle two if n is even)

Table 3.1: Math Service output fields

3.2.3 Source Code

```
1 #!/usr/bin/env python3
2 """Python Statistical Computation Microservice via HTTP."""
```

```

3 from http.server import HTTPServer, BaseHTTPRequestHandler
4 import json
5
6 class MathHandler(BaseHTTPRequestHandler):
7     def do_POST(self):
8         length = int(self.headers.get('Content-Length', 0))
9         body = json.loads(self.rfile.read(length))
10        data = body.get('data', [])
11
12        if not data:
13            result = {"error": "La liste de donnees est vide"}
14            status = 400
15        else:
16            sorted_data = sorted(data)
17            n = len(sorted_data)
18
19            # Statistical computations
20            minimum = sorted_data[0]
21            maximum = sorted_data[-1]
22            moyenne = sum(sorted_data) / n
23            mediane = (sorted_data[n // 2] if n % 2 != 0
24                       else (sorted_data[n // 2 - 1] +
25                             sorted_data[n // 2]) / 2)
26
27            result = {
28                "count": n,
29                "min": minimum,
30                "max": maximum,
31                "moyenne": moyenne,
32                "mediane": mediane,
33            }
34            status = 200
35
36            self.send_response(status)
37            self.send_header('Content-Type', 'application/json')
38            self.end_headers()
39            self.wfile.write(json.dumps(result).encode())
40
41        def log_message(self, format, *args):
42            pass
43
44    if __name__ == '__main__':
45        server = HTTPServer(('0.0.0.0', 8081), MathHandler)
46        print('Math Service started on port 8081')
47        server.serve_forever()

```

Listing 3.3: math_service.py

3.2.4 Dockerfile

```

1 FROM python:3.11-slim

```

```
2
3 LABEL maintainer='G04@ensa-tetouan.ma'
4 LABEL version='1.0'
5 LABEL description='Microservice de calcul statistique Python'
6
7 WORKDIR /app
8
9 COPY math_service.py .
10
11 EXPOSE 8081
12
13 RUN useradd -m appuser && chown -R appuser /app
14 USER appuser
15
16 CMD ["python", "math_service.py"]
```

Listing 3.4: Math Service Dockerfile

3.3 API Gateway Service

3.3.1 Responsibility

The Gateway is the **only service exposed to external clients**. It orchestrates the two back-end services in parallel fan-out fashion:

1. Forwards the raw request body to the Sort Service.
2. Forwards the same raw request body to the Math Service.
3. Aggregates both responses into a single JSON reply.

If either back-end is unreachable, the Gateway returns an HTTP 502 **Bad Gateway** error with a descriptive message.

API Contract

Endpoint: POST / **Port:** 8000 (container) / 82 (cluster) / 30082 (node)

Request body:

```
{ "data": [5, 3, 8, 1, 2] }
```

Successful response (HTTP 200):

```
{
  "status": "success",
  "sorted_data": [1, 2, 3, 5, 8],
  "statistics": {
    "count": 5,
    "min": 1,
    "max": 8,
    "moyenne": 3.8,
  }
}
```

```

    "mediane": 3.0
  }
}

```

Error response (HTTP 502):

```

{
  "status": "error",
  "message": "Failed to communicate with backend services:
    ..."
}

```

3.3.2 Source Code

```

1  #!/usr/bin/env python3
2  """API Gateway Microservice - Entrypoint for the cluster."""
3  from http.server import HTTPServer, BaseHTTPRequestHandler
4  import json
5  import urllib.request
6  import urllib.error
7
8  class GatewayHandler(BaseHTTPRequestHandler):
9      def do_POST(self):
10         length = int(self.headers.get('Content-Length', 0))
11         body_data = self.rfile.read(length)
12
13         try:
14             # Step 1: Contact the Sorting Service via K8s DNS
15             sort_url = "http://web-app-service:80"
16             req_sort = urllib.request.Request(
17                 sort_url,
18                 data=body_data,
19                 headers={'Content-Type': 'application/json'},
20                 method='POST'
21             )
22             with urllib.request.urlopen(req_sort) as response:
23                 sorted_res =
24                     json.loads(response.read().decode())
25
26             # Step 2: Contact the Math Service via K8s DNS
27             math_url = "http://math-service-service:81"
28             req_math = urllib.request.Request(
29                 math_url,
30                 data=body_data,
31                 headers={'Content-Type': 'application/json'},
32                 method='POST'
33             )
34             with urllib.request.urlopen(req_math) as response:
35                 math_res = json.loads(response.read().decode())
36
37             # Step 3: Aggregate results

```

```

37         final_response = {
38             "status": "success",
39             "sorted_data": sorted_res.get("sorted", []),
40             "statistics": math_res
41         }
42         status_code = 200
43
44     except urllib.error.URLError as e:
45         final_response = {
46             "status": "error",
47             "message": f"Failed to communicate with backend
48                 services: {str(e)}"
49         }
50         status_code = 502
51     except Exception as e:
52         final_response = {"status": "error", "message":
53             str(e)}
54         status_code = 500
55
56     self.send_response(status_code)
57     self.send_header('Content-Type', 'application/json')
58     self.end_headers()
59     self.wfile.write(json.dumps(final_response).encode())
60
61     def log_message(self, format, *args):
62         pass # Suppress access logs
63
64 if __name__ == '__main__':
65     server = HTTPServer(('0.0.0.0', 8000), GatewayHandler)
66     print('API Gateway running on port 8000')
67     server.serve_forever()

```

Listing 3.5: gateway_service.py

3.3.3 Dockerfile

```

1 FROM python:3.11-slim
2
3 LABEL maintainer='G04@ensa-tetouan.ma'
4 LABEL version='1.0'
5 LABEL description='API Gateway Ingestion Service'
6
7 WORKDIR /app
8
9 COPY gateway_service.py .
10
11 EXPOSE 8000
12
13 RUN useradd -m appuser && chown -R appuser /app
14 USER appuser
15

```

```
16 CMD ["python", "gateway_service.py"]
```

Listing 3.6: Gateway Service Dockerfile

Chapter 4

Kubernetes Configuration Files

4.1 Sort Service — Deployment & Service (deployment.yml and service.yml)

4.1.1 Deployment

```
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: web-app-deployment
5   labels:
6     app: sort-service
7 spec:
8   replicas: 2           # Two replicas for high availability
9   strategy:
10    type: RollingUpdate
11    rollingUpdate:
12      maxSurge: 1        # At most 1 extra pod during update
13      maxUnavailable: 0 # Zero downtime: no pod removed before
                          # new one is ready
14   selector:
15     matchLabels:
16       app: sort-service
17   template:
18     metadata:
19       labels:
20         app: sort-service
21     spec:
22       containers:
23         - name: sort-container
24           image: g04/sort-service:1.0
25           imagePullPolicy: IfNotPresent
26       ports:
27         - containerPort: 8080
28       resources:
```



```

29     requests:
30         memory: "64Mi"
31         cpu: "100m"
32     limits:
33         memory: "128Mi"
34         cpu: "250m"

```

Listing 4.1: k8s/deployment.yml — Sort Service Deployment

4.1.2 Service

```

1  apiVersion: v1
2  kind: Service
3  metadata:
4      name: web-app-service
5  spec:
6      type: NodePort
7      selector:
8          app: sort-service # Routes to pods labelled sort-service
9      ports:
10         - protocol: TCP
11           port: 80 # Internal cluster port
12           targetPort: 8080 # Maps to the Python container port
13           nodePort: 30080 # Accessible from the host machine

```

Listing 4.2: k8s/service.yml — Sort Service

4.1.3 Resource Limits Explained

Resource	Request	Limit	Meaning
Memory	64 MiB	128 MiB	Pod will be scheduled only if 64 MiB is available; killed if it exceeds 128 MiB
CPU	100 m	250 m	Guaranteed 0.1 core; capped at 0.25 core

Table 4.1: Sort Service resource constraints

4.2 Math Service — Deployment & Service (math-deployment.yml)

```

1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4      name: math-service-deployment
5      labels:
6          app: math-service
7  spec:
8      replicas: 2

```

```
9   selector:
10     matchLabels:
11       app: math-service
12   template:
13     metadata:
14       labels:
15         app: math-service
16     spec:
17       containers:
18         - name: math-container
19           image: g04/math-service:1.0
20           imagePullPolicy: IfNotPresent
21       ports:
22         - containerPort: 8081
23 ---
24 apiVersion: v1
25 kind: Service
26 metadata:
27   name: math-service-service
28 spec:
29   type: NodePort
30   selector:
31     app: math-service
32   ports:
33     - protocol: TCP
34       port: 81
35       targetPort: 8081
36       nodePort: 30081
```

Listing 4.3: k8s/math-deployment.yml

4.3 Gateway Service — Deployment & Service (gateway-deployment.yml)

```
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: gateway-deployment
5   labels:
6     app: gateway-service
7 spec:
8   replicas: 2
9   selector:
10     matchLabels:
11       app: gateway-service
12   template:
13     metadata:
14       labels:
15         app: gateway-service
16   spec:
```

```
17     containers:
18     - name: gateway-container
19       image: g04/gateway-service:1.0
20       imagePullPolicy: IfNotPresent
21     ports:
22     - containerPort: 8000
23 ---
24 apiVersion: v1
25 kind: Service
26 metadata:
27   name: gateway-service
28 spec:
29   type: NodePort
30   selector:
31     app: gateway-service
32   ports:
33   - protocol: TCP
34     port: 82
35     targetPort: 8000
36     nodePort: 30082
```

Listing 4.4: k8s/gateway-deployment.yml

Chapter 5

Docker Images & DockerHub

5.1 Published Images

All three images are published to the DockerHub account `marouaneloua`:

Image	Full Pull Reference
Sort Service	<code>docker pull marouaneloua/sort-service:1.0</code>
Math Service	<code>docker pull marouaneloua/math-service:1.0</code>
Gateway Service	<code>docker pull marouaneloua/gateway-service:1.0</code>

Table 5.1: DockerHub image references

5.2 Building Images Locally (Minikube)

When deploying to Minikube and building images locally (without pushing to DockerHub), use:

```
minikube image build -t g04/sort-service:1.0 ./src/sort-service
minikube image build -t g04/math-service:1.0 ./src/math-service
minikube image build -t g04/gateway-service:1.0 ./src/gateway-service
```

Listing 5.1: Build images directly inside Minikube's Docker daemon

5.3 Building and Pushing to DockerHub

```
# Authenticate
docker login

# Sort Service
docker build -t marouaneloua/sort-service:1.0 ./src/sort-service
docker push marouaneloua/sort-service:1.0

# Math Service
docker build -t marouaneloua/math-service:1.0 ./src/math-service
docker push marouaneloua/math-service:1.0

# Gateway Service
docker build -t marouaneloua/gateway-service:1.0 ./src/gateway-service
docker push marouaneloua/gateway-service:1.0
```

Listing 5.2: Build and push to DockerHub as marouaneloua

When using DockerHub images in Kubernetes manifests, update the `image:` field to the full qualified name, e.g. `marouaneloua/sort-service:1.0`, and set `imagePullPolicy:` `Always` or `IfNotPresent`.

Chapter 6

Deployment Guide

6.1 Prerequisites

- **Docker** (tested with 24.x+)
- **Minikube** (tested with 1.33+)
- **kubectl** (matches your cluster version)
- Sufficient resources: at least **2 CPU cores** and **2 GB RAM** for Minikube

6.2 Step-by-Step Deployment

6.2.1 Step 1 — Start Minikube

```
minikube start
```

6.2.2 Step 2 — Option A: Build Images in Minikube (local)

```
# Clone / unzip the project first, then:
minikube image build -t g04/sort-service:1.0    ./src/sort-
service
minikube image build -t g04/math-service:1.0    ./src/math-
service
minikube image build -t g04/gateway-service:1.0 ./src/gateway-
service
```

6.2.3 Step 2 — Option B: Pull Images from DockerHub

Update all `image:` fields in the `k8s/` YAML files to use the full DockerHub path, then proceed to Step 3.

```
# Example: update sort-service image reference
sed -i 's|g04/sort-service:1.0|marouaneloua/sort-service:1.0|g'
    k8s/deployment.yml
sed -i 's|g04/math-service:1.0|marouaneloua/math-service:1.0|g'
    k8s/math-deployment.yml
sed -i 's|g04/gateway-service:1.0|marouaneloua/gateway-service
:1.0|g' k8s/gateway-deployment.yml
```

6.2.4 Step 3 — Apply All Kubernetes Manifests

```
kubectl apply -f k8s/
```

6.2.5 Step 4 — Verify Deployment

```
# Check all pods are Running
kubectl get pods

# Check services
kubectl get services

# Watch rollout status
kubectl rollout status deployment/web-app-deployment
kubectl rollout status deployment/math-service-deployment
kubectl rollout status deployment/gateway-deployment
```

6.2.6 Step 5 — Get the Gateway URL

```
minikube service gateway-service --url
# Output example: http://192.168.49.2:30082
```

6.2.7 Step 6 — Test the API

```
# Replace <URL> with the output of the previous command
curl -X POST http://<URL>/ \
    -H "Content-Type: application/json" \
    -d '{"data": [42, 7, 19, 3, 88, 1]}'
```

Expected response:

```
{
  "status": "success",
  "sorted_data": [1, 3, 7, 19, 42, 88],
  "statistics": {
    "count": 6,
    "min": 1,
    "max": 88,
    "moyenne": 26.666666666666668,
```

```
    "mediane": 13.0  
  }  
}
```

6.3 Tearing Down

```
# Remove all deployed resources  
kubectl delete -f k8s/  
  
# Stop Minikube  
minikube stop
```


Chapter 7

Kubernetes Features in Use

7.1 High Availability via Replicas

Every service is configured with **2 replicas**. Kubernetes schedules these replicas across available nodes. If one pod crashes, the other continues serving traffic while Kubernetes automatically recreates the failed pod.

7.2 Rolling Updates (Zero Downtime)

The Sort Service Deployment explicitly configures a `RollingUpdate` strategy:

- `maxSurge: 1` — Kubernetes may create one extra pod above the desired count.
- `maxUnavailable: 0` — No pod is terminated until a new, healthy pod is ready.

This guarantees zero-downtime deployments when the image is updated.

7.3 Resource Quotas

The Sort Service defines both `requests` (scheduling guarantee) and `limits` (hard cap):

- The scheduler uses *requests* to find a node with enough available capacity.
- The kubelet enforces *limits*; a container exceeding its memory limit is OOM-killed.

7.4 Internal DNS-Based Service Discovery

Rather than hardcoded IP addresses, the Gateway uses Kubernetes DNS:

- `http://web-app-service:80` — resolves to the Sort Service ClusterIP.

- `http://math-service-service:81` — resolves to the Math Service ClusterIP.

This decouples services from topology changes and makes the configuration portable.

7.5 NodePort Services

Each service is exposed as a **NodePort**, meaning Kubernetes opens a static port on every node in the range 30000-32767. In Minikube this corresponds to a port on the VM/container running the cluster, accessible from the host via `minikube service <name> -url`.

Chapter 8

Security Considerations

8.1 Non-Root Container Execution

All three Dockerfiles create a dedicated `appuser` account and switch to it before starting the application:

```
RUN useradd -m appuser && chown -R appuser /app
USER appuser
```

Running as a non-root user prevents a compromised container from writing to host-mounted paths or escalating privileges via kernel exploits that require root.

8.2 Minimal Base Image

All images use `python:3.11-slim`, a Debian slim variant that excludes compilers, package managers, and other tooling not required at runtime, significantly reducing the number of potential CVEs.

8.3 No External Dependencies

The Sort and Math services use only Python's standard library (`http.server`, `json`). The Gateway service also uses only the standard library. This eliminates supply-chain risks from third-party packages.

8.4 Internal-Only Back-End Services

The Sort and Math services are internal to the cluster; they have NodePort services configured but are not intended to receive direct external traffic. In a production environment they should be changed to `ClusterIP` type to prevent direct external access.

Chapter 9

Project File Structure

```
G04_Projet_Kubernetes/  
|-- README.md  
|-- k8s/  
|   |-- deployment.yml           # Sort Service Deployment  
|   |-- service.yml              # Sort Service Service  
|   |-- math-deployment.yml      # Math Service Deployment +  
|       Service  
|   '-- gateway-deployment.yml   # Gateway Deployment + Service  
'-- src/  
    |-- sort-service/  
    |   |-- Dockerfile  
    |   |-- requirements.txt      # (empty - stdlib only)  
    |   '-- sort_service.py  
    |-- math-service/  
    |   |-- Dockerfile  
    |   '-- math_service.py  
    '-- gateway-service/  
        |-- Dockerfile  
        '-- gateway_service.py
```

Listing 9.1: Repository layout

Chapter 10

Troubleshooting

10.1 Common Issues

10.1.1 Pods in ImagePullBackOff

Cause: Kubernetes cannot pull the image. Either the image name is wrong or it is not available in the registry.

Fix: Verify the image name and `imagePullPolicy`. If using local Minikube images, ensure you built them inside Minikube's Docker daemon (`minikube image build`), not on the host. Alternatively pull from DockerHub using the full `marouaneloua/` prefix.

10.1.2 Gateway Returns 502

Cause: The gateway cannot reach one of the back-end services.

Fix: Check that the Sort and Math Service pods are running (`kubectl get pods`) and that their Services are correctly named `web-app-service` and `math-service-service`.

10.1.3 Minikube Tunnel Not Accessible

Cause: `minikube service gateway-service -url` hangs or returns an unreachable address.

Fix: Run `minikube tunnel` in a separate terminal, or use the direct NodePort with the Minikube IP: `minikube ip` then `curl http://<minikube-ip>:30082/`.

10.2 Useful Diagnostic Commands

```
# List all pods with status
kubectl get pods -o wide
```

```
# Describe a specific pod (events, errors)
kubectl describe pod <pod-name>

# View logs of a running container
kubectl logs deployment/gateway-deployment
kubectl logs deployment/web-app-deployment
kubectl logs deployment/math-service-deployment

# Interactive shell inside a pod for network debugging
kubectl exec -it <pod-name> -- /bin/sh

# Test internal DNS from inside the cluster
kubectl run tmp --image=python:3.11-slim --rm -it --restart=
Never \
-- python3 -c "import urllib.request; \
    print(urllib.request.urlopen('http://web-app-service:80').
        read())"
```

Chapter 11

Summary

This project successfully implements a **cloud-native microservices application** orchestrated with Kubernetes. The key achievements are:

- **Separation of concerns:** Each service performs exactly one task (sorting or statistics), making them independently testable, deployable, and scalable.
- **Zero-downtime updates:** Rolling update strategy with `maxUnavailable: 0` on the Sort Service.
- **Security:** Non-root container users and minimal `slim` base images across all services.
- **No runtime dependencies:** Pure Python standard library — no pip packages required at runtime for Sort and Math services.
- **Kubernetes-native networking:** Internal DNS eliminates hardcoded IPs and enables seamless pod rescheduling.
- **Resource governance:** CPU and memory requests/limits prevent noisy-neighbour problems.
- **Reproducible deployment:** A single `kubectl apply -f k8s/` command recreates the entire infrastructure.